ISM2411 — Lab Week 04

Revenue, Margin & Discount Calculator — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 04 · Unit 1 · Foundations

Contents

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 04 Lab — Revenue, Margin & Discount Calculator
Unit	Unit 1 · Foundations
Class length	75 minutes
Format	Live code-along — you type on the shared screen, students type along on their own machines and run every step with you
Prerequisites	Modules 01–03: file system basics, variables and data types, `print()` / f-strings
Student-facing lab page	markumreed.github.io/ism2411 — week04_lab
Exercises covered	Exercises 1–7 (required) + Stretch A/B (as time allows)
Submission	`calculator.py` to Canvas, all seven exercises in one file, each marked with a `# --- Exercise N ---` comment

This is the first lab this semester where students write real, graded business logic — arithmetic operators, comparisons, and boolean logic applied to pricing, margin, and discount formulas. Modules 1–3 were about reading and typing things correctly; this module is about **computing** things correctly, and about the specific ways Python's operators quietly do something other than what a spreadsheet-trained brain expects (integer vs. float division, operator precedence, percentage-as-decimal). Budget real time for the mistakes — they are the content, not a distraction from it.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Write arithmetic expressions using `+ - * / // %` and correctly predict what each one returns, including the float-vs-integer distinction between `/` and `//`.
2. Use comparison operators (`> < == >=` etc.) to produce a boolean value and explain what a boolean *is* (not a string, not a number).
3. Combine two boolean conditions with `and` / `or` and correctly reason about the resulting truth value.
4. Format numeric output for a business audience using f-string format specs: `,.2f` for currency, `.1%` for percentages.
5. Read a multi-line business calculation and identify where a missing parenthesis would silently produce a wrong (but plausible-looking) answer — this is the single highest-value takeaway of the day.
6. Collect multiple values from a user with `input()`, convert them to the correct numeric type, and use them in a downstream calculation.

3 Before Class — Setup Checklist

- ☐ Open a Python editor you can screen-share (VS Code recommended — matches what students installed in Module 01). Font size large enough to read from the back row (18–20pt).
- ☐ Open a terminal pane next to the editor so output is visible without alt-tabbing.
- ☐ Pre-create an empty `calculator.py` file — do **not** pre-write the exercises. Everything below is meant to be typed live, in front of the class, mistakes included.
- ☐ Test every code block in this guide against your classroom's Python version before class (this guide was verified against Python 3.12; all output is exact and reproducible — see the answer key in the Appendix).
- ☐ Pull up the student-facing lab page on a second tab so you can point students to the exact exercise text and "Expected" output when framing each exercise.
- ☐ Decide in advance which two or three students you'll cold-call for Exercise 5 (the `/ // %` prediction) — cold-calling *before* running the code is what makes that exercise land.

4 Materials Needed

- Instructor laptop + projector/screen-share, Python 3.10+ installed
- Students: their own laptops, `calculator.py` open in the same editor they used in Modules 1–3
- No new libraries, no internet access required — this entire lab is pure Python built-ins

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:05	Welcome, recap Modules 1–3, frame today's file	5
0:05–0:15	Exercise 1 (Revenue) + Exercise 2 (Margin)	10
0:15–0:22	Exercise 3 (Two products / comparison)	7
0:22–0:30	Exercise 4 (Logical combo / `and`)	8
0:30–0:40	Exercise 5 (`/` vs `//` vs `%` prediction)	10
0:40–0:55	Exercise 6 (Full pricing calculator with `input()`)	15
0:55–1:07	Exercise 7 (Packaging problem)	12
1:07–1:15	Stretch A/B as time allows + wrap-up, reflection, submission checklist	8

This plan already uses all 75 minutes on the seven required exercises plus buffer — there is no thin spot to pad here. If your section moves faster than this pacing (common with an experienced group), the two Stretch challenges and the "Extra Practice" block in the Appendix are the release valve; do **not** cut required exercises short to reach the stretch material.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:05)

Say to the class:

> *"Modules 1 through 3 got you reading file paths, storing values in variables, and printing them out formatted. Today we start actually* computing *things — and we're doing it with real business formulas: revenue, margin, discounts. Two warnings before we start. One: Python's division operators do not work the way your calculator does — `/` and `//` give you different types of answers, and that difference matters. Two: today's single most important idea is that a missing parenthesis doesn't crash your program — it just quietly gives you the wrong number. Nobody tells you. Your boss doesn't tell you. The spreadsheet doesn't tell you. You have to catch it yourself. That's what today is really about."*

Do: Open `calculator.py`. Type the file header comment together:

```
# calculator.py
# ISM2411 Module 04 Lab — Revenue, Margin & Discount Calculator
```

Tell students this comment convention — one header line, plus a `# --- Exercise N ---` comment above every exercise — is required for submission and will be required in every script for the rest of the semester.

6.2 Exercise 1 — Revenue (0:05–0:10, 5 min)

Teaching goal: Multiplication as the business operation "price times quantity," and the first f-string format spec of the semester: thousands separator + fixed decimals.

Say to the class:

> *"Revenue is the simplest formula in the whole lab — price times quantity — but the* output formatting *is the actual skill we're practicing. `$600` and `$600.00` mean the same number to Python, but only one of them looks like something a manager would accept in a report."*

Live-code this, line by line:

```
# --- Exercise 1 ---
price = 50
quantity = 12
revenue = price * quantity
print(f"Revenue: ${revenue:,.2f}")
```

Line-by-line explanation:

- `price = 50` — an integer variable. Point out: no dollar sign, no comma, no quotes — just the raw number. Typing `price = "$50"` would make this a string, and `"$50" * 12` would not do what students expect (it would repeat the text). This is worth a 15-second aside.
- `quantity = 12` — likewise a plain integer.
- `revenue = price * quantity` — `*` is the multiplication operator. `price` and `quantity` are both `int`, so `revenue` is also an `int` (`600`), not a float yet.
- `print(f"Revenue: ${revenue:,.2f}")` — this is the line to slow down on:
  - The `f` before the opening quote makes this an **f-string** — anything inside `{ }` is Python code, evaluated and substituted into the string.
  - The literal `$` right before `{revenue...}` is just a character in the string — Python does not interpret it, it's printed as-is.
  - Inside the braces: `revenue` is the value being formatted; everything after the `:` is the **format spec**.
  - `,` inserts a thousands separator (`1200` becomes `1,200`) — not needed for `600` here, but essential once revenue crosses 1,000, which it will in Exercise 6.
  - `.2f` means "fixed-point notation, 2 digits after the decimal" — this is what turns the integer `600` into the display string `600.00`. Emphasize: this doesn't change the *stored* value of `revenue`, only how it's displayed by this one `print()` call.

Run it. Expected output:

```
Revenue: $600.00
```

Common student mistakes to watch for:

- Forgetting the `f` before the string — the braces print literally as `{revenue:,.2f}` instead of substituting. This is the most common error in the room; walk by and check for it.
- Putting the format spec inside quotes, e.g. `f"Revenue: ${revenue}:,.2f"` — the colon has to be *inside* the braces.
- Confusing `,.2f` order — students sometimes write `.2f,` and get a `ValueError`. Show one deliberately broken version and let them see the traceback; the error message actually says `Invalid format specifier`, which is worth reading aloud once.

Check for understanding: Ask the room: "If I change `quantity` to `12.5`, does this line still work, and does the output still make sense?" (Yes — `revenue` becomes a float, `,.2f` still formats it correctly. This previews that `,.2f` works on both ints and floats, which matters in Exercise 6.)

6.3 Exercise 2 — Margin (0:10–0:15, 5 min)

Teaching goal: True division (`/`) always returns a float, and the `.1%` format spec — the first time students see Python multiply a decimal by 100 and add a `%` sign automatically.

Say to the class:

> *"Margin is the percentage of the price that's profit. The formula is `(price - cost) / price`. Watch what happens with the formatting here — we are* not *going to multiply by 100 ourselves."*

Live-code this:

```
# --- Exercise 2 ---
cost = 32
price = 50
margin = (price - cost) / price
print(f"Margin: {margin:.1%}")
```

Line-by-line explanation:

- `cost = 32`, `price = 50` — reusing `price` here shadows the value from Exercise 1. Point this out explicitly: **variables are not scoped per-exercise** — once `price` is reassigned to `50` again here it's fine because it's the same value, but flag that in Exercise 3 we'll rename to `price_a` / `price_b` specifically to avoid this kind of collision.
- `margin = (price - cost) / price` — the parentheses are **required**, not stylistic. `price - cost / price` would divide `cost` by `price` first (division binds tighter than subtraction) and then subtract — a completely different, wrong number. This is the first live preview of the precedence trap that Stretch B drills on directly; flag it now so it's not a surprise later.
- `/` is **true division** — it always returns a `float`, even when the answer would come out evenly (e.g., `10 / 2` is `5.0`, not `5`). Contrast this verbally with `//`, which is coming in Exercise 5.
- `margin` is `0.36` — a *decimal fraction*, not "36." This is the detail the format spec handles for you next.
- `f"Margin: {margin:.1%}"` — the `%` format type does two things at once: multiplies the value by 100, and appends a literal `%` character. `.1` means one digit after the decimal point *in the percentage*, not in the raw fraction. So `0.36` becomes `36.0%`.

Run it. Expected output:

```
Margin: 36.0%
```

Common student mistakes to watch for:

- Manually multiplying by 100 first (`margin = (price - cost) / price * 100`) and *then* using `.1%`, which double-multiplies and gives `3600.0%`. This is extremely common — walk the room and watch for it. When you spot it, don't just correct it; ask the student to explain out loud why the number is 100x too big. Making them state "because `%` already multiplies by 100" is what makes it stick.
- Omitting the parentheses around `price - cost`, discussed above.

Check for understanding: Cold-call: "What data type is `margin` right now — before it's ever printed?" (A `float`, `0.36`. The `%` in the f-string changes *display*, not the stored type — reinforce

this distinction, it recurs in Exercise 6.)

6.4 Exercise 3 — Two Products (0:15–0:22, 7 min)

Teaching goal: Comparison operators produce a `bool`, and a `bool` is a real, printable value — not a string, not a special case.

Say to the class:

> *"Now we compare two products by margin. Notice I'm renaming variables this time — `price_a, cost_a, price_b, cost_b` — because we just saw what happens when you reuse a bare `price` for two different things. This exercise also introduces the `bool` type: `True` and `False`, capitalized, no quotes."*

Live-code this:

```
# --- Exercise 3 ---
price_a, cost_a = 50, 32
price_b, cost_b = 80, 60
margin_a = (price_a - cost_a) / price_a
margin_b = (price_b - cost_b) / price_b
product_a_wins = margin_a > margin_b
print(f"Product A margin: {margin_a:.1%}")
print(f"Product B margin: {margin_b:.1%}")
print(f"Product A wins: {product_a_wins}")
```

Line-by-line explanation:

- `price_a, cost_a = 50, 32` — this is **tuple unpacking**: one line assigns two variables at once, left to right. Point out this is optional shorthand for two separate `price_a = 50` / `cost_a = 32` lines — either style is fine, but this is a good moment to show it exists since students will see it in later modules.
- `margin_a = (price_a - cost_a) / price_a` and the matching line for `margin_b` — identical formula from Exercise 2, applied to each product independently. Ask the room to predict both margins before running (36.0% and 25.0% — Product B has a lower margin despite a higher price, which is itself worth a 10-second business aside: higher price does not automatically mean higher margin).
- `product_a_wins = margin_a > margin_b` — `>` is a **comparison operator**. This line does not print anything and does not branch anything — it evaluates `margin_a > margin_b` to either `True` or `False` and stores *that* value in `product_a_wins`. This is the core idea to land here: a comparison is an expression that produces a value, exactly like `2 + 2` produces `4`.
- `print(f"Product A wins: {product_a_wins}")` — `product_a_wins` is a `bool`, and f-strings print booleans as the literal words `True` or `False` (capital T/F, no quotes). Emphasize: `True` is not the string `"True"` — if a student later writes `if product_a_wins == "True":` it will silently never match, because a bool is never equal to a string. This is worth stating explicitly now even though `if` doesn't appear until Module 05 — it prevents a very common bug two weeks from now.

Run it. Expected output:

```
Product A margin: 36.0%
Product B margin: 25.0%
Product A wins: True
```

Common student mistakes to watch for:

- Using = instead of ==… they haven't hit == yet in this exercise (that's Exercise 4), but if a student tries to write `product_a_wins = margin_a = margin_b` out of confusion, catch it — that's an assignment chain, not a comparison, and it will silently overwrite `margin_a` and `margin_b` both to whatever `margin_b` was. Good moment to distinguish = (assignment, "put this value in this box") from > < == (comparison, "ask a true/false question").

Check for understanding: "If Product B's margin had been *higher* than Product A's, what would `product_a_wins` print?" (`False` — make someone say it out loud, and make someone else explain that the variable name doesn't change what's stored in it; `product_a_wins` is just a label, it will happily hold `False`.)

6.5 Exercise 4 — Logical Combo (0:22–0:30, 8 min)

Teaching goal: Combining two boolean conditions with `and`, and observing how changing one input changes the combined result.

Say to the class:

> *"A 'premium' product needs to satisfy* two *conditions at once: price over 100 AND margin over 30%. This is where `and` comes in — it's not addition, it's a logical operator that only returns `True` when both sides are `True`."*

Live-code this:

```python
# --- Exercise 4 ---
price = 120
margin = 0.35
is_premium = price > 100 and margin > 0.3
print(f"is_premium = {is_premium}")
```

Line-by-line explanation:

- `price = 120`, `margin = 0.35` — margin is entered directly as a decimal here (not computed), since this exercise is isolating the logic, not the formula.
- `is_premium = price > 100 and margin > 0.3` — read this out loud exactly as Python evaluates it: first `price > 100` is evaluated on its own (`120 > 100` → `True`), then `margin > 0.3` is evaluated on its own (`0.35 > 0.3` → `True`), and only *then* does `and` combine the

two booleans (`True and True` → `True`). Students commonly try to read this as one big arithmetic-style expression; walk it in three explicit steps on the board/screen.
- `and` truth table — say it once, explicitly: `True and True` is `True`; any other combination (`True and False`, `False and True`, `False and False`) is `False`. Both sides must be true.

Run it. Expected output:

```
is_premium = True
```

Now, live, change the values so exactly one condition is met:

```
price = 120
margin = 0.20
is_premium = price > 100 and margin > 0.3
print(f"is_premium = {is_premium}")
```

Run it — output is `is_premium = False`, even though `price > 100` alone is `True`. This is the entire point of the exercise: say explicitly, "One true condition is not enough for `and`. Both have to be true." Then try the reverse (`price = 80, margin = 0.35`) and get `False` again, for the same reason from the other side.

Common student mistakes to watch for:

- Writing `price > 100 and margin > 30` (forgetting margin is a decimal, not a whole-number percentage) — this compiles and runs without error, just always evaluates unexpectedly. This is a "silent wrong answer" bug, exactly the category the intro warned about — flag it as such when you see it.
- Confusing `and` with `&` — `&` exists in Python but is bitwise, not logical, and will misbehave here. Not worth a deep detour, but worth a one-sentence "we're not using that operator today" if someone tries it.

Check for understanding: "I want to check if a product is premium OR on clearance — would I use `and` here?" (No — `or`, which returns `True` if *either* side is true. Don't code it yet — that's a natural bridge into Stretch B and into next module's `if`/`elif` — just get the room to say "or" out loud correctly.)

6.6 Exercise 5 — Common-Mistake Check: `/`, `//`, `%` (0:30–0:40, 10 min)

Teaching goal: This is the highest-leverage exercise in the lab. Students must be able to distinguish true division, floor division, and modulo — and this distinction underpins Exercise 7 and half of next semester's logic.

Say to the class:

> *"Before we run anything — everyone write down, on paper or in a comment, your prediction for `17 / 5`, `17 // 5`, and `17 % 5`. I will cold-call three of you before we run this."*

Do this as a genuine predict-then-verify exercise — do not type the code until predictions are collected. Cold-call 2–3 students for their predicted values. Common wrong predictions: `17 // 5` predicted as `3.4` (confusing it with `/`), `17 % 5` predicted as some kind of percentage (`%` here is the *modulo* operator, unrelated to the `.1%` format spec from Exercise 2 — flag that these are two completely different uses of the same character, worth stating explicitly since it's a legitimate source of confusion).

Live-code this:

```
# --- Exercise 5 ---
print(17 / 5)    # true division
print(17 // 5)   # floor division
print(17 % 5)    # modulo (remainder)
```

Line-by-line explanation:

- `17 / 5` — true division, always returns a `float`: `3.4`. This is "what a calculator gives you."
- `17 // 5` — floor division: divide, then round *down* to the nearest whole number, discarding the fractional part entirely (not rounding to nearest — always toward negative infinity). Result: `3`. With two positive integers this returns an `int`; note for the curious that `//` on floats returns a float that's still a whole number, e.g. `17.0 // 5` is `3.0`.
- `17 % 5` — modulo: the **remainder** left over after floor division. `17 = 5 × 3 + 2`, so the remainder is `2`. The connection to state explicitly: `//` and `%` are a matched pair — `(a // b) * b + (a % b)` always reconstructs `a`. Write that identity on the board; it's the single sentence that makes Exercise 7 make sense.

Run it. Expected output:

```
3.4
3
2
```

Now, have each student write one sentence per operator explaining what it does and when they'd use it in a business calculation (this is required by the exercise itself — give them 2 minutes of quiet writing time here, it's built into the 10-minute budget). Prompt if the room is stuck:

"`//` and `%` show up anywhere you're packing items into fixed-size groups — boxes, pallets, pages, shifts." That's a direct setup for Exercise 7.

Common student mistakes to watch for:

- Assuming `//` rounds to the *nearest* integer rather than always down. Show a negative example if time allows: `-7 // 2` is `-4`, not `-3`, which surprises almost everyone (floor division rounds toward negative infinity, not toward zero). Optional aside if pacing allows; skip it if you're behind.
- Confusing `%` (modulo, works on numbers) with `%` in an f-string format spec (multiplies by 100 for display). Say explicitly: same character, two unrelated meanings depending on context — inside a format spec after a colon, it's display formatting; as an operator between two numbers, it's modulo.

Check for understanding: "If I have 50 items and box size 8, what does `50 // 8` give me, and what does `50 % 8` give me — without running it?" (`6` full boxes, `2` left over. Have someone verify by hand: `8 × 6 = 48`, `50 − 48 = 2`. This hand-verification is exactly what Exercise 7 automates.)

6.7 Exercise 6 — Full Pricing Calculator (0:40–0:55, 15 min)

Teaching goal: Collecting multiple inputs with `input()`, converting text to numbers with `float()`, and chaining several of today's formulas into one coherent report. This is the exercise students will spend the most time on independently — go a little slower here and leave room for the room to type along in real time rather than just watch.

Say to the class:

> *"Everything up to now used values we typed directly into the script. Real tools ask the* user *for values. `input()` always returns a string — even if someone types `50` — so every numeric input has to be explicitly converted, or your very first calculation will crash."*

Live-code this in three passes — first just the inputs, then the calculations, then the formatted report. Do not type the whole block at once; build it up so students see the reasoning, not just the destination.

Pass 1 — collect inputs:

```
# --- Exercise 6 ---
product_name = input("Product name: ")
retail_price = float(input("Retail price: "))
unit_cost = float(input("Unit cost: "))
quantity_sold = int(input("Quantity sold: "))
discount_percent = float(input("Discount percent (e.g. 10 for 10%): "))
```

Line-by-line explanation:

- `product_name = input("Product name: ")` — `input()` displays the prompt string, pauses execution, and returns whatever the user typed **as a string**, always — even `"340"` typed for a quantity comes back as the string `"340"`, not the number `340`. `product_name` genuinely is a string, so no conversion needed here.
- `retail_price = float(input("Retail price: "))` — read this inside-out: `input(...)` runs first and returns a string (e.g. `"29.99"`), and `float(...)` wraps that call and converts the string to a `float`. This is function composition — one function's return value fed directly into another — worth naming explicitly since it looks dense the first time.
- `unit_cost = float(...)` — same pattern.
- `quantity_sold = int(input(...))` — this one uses `int()`, not `float()`, because a quantity should be a whole number. Flag the failure mode: if a student types `340.5` here, `int("340.5")` raises a `ValueError` — `int()` cannot parse a string containing a decimal point directly. This is a deliberately good moment to run it live and let the class see the traceback.
- `discount_percent = float(...)` — collected as a whole number like `10`, **not** as `0.10` — say explicitly that the conversion to a decimal fraction happens in the next block, not here.

Run pass 1 alone with sample input (`Wireless Mouse`, `29.99`, `11.50`, `340`, `15`) and add one line — `print(product_name, retail_price, unit_cost, quantity_sold, discount_percent)` — so the room can see the five values landed correctly before layering on the math. Remove that debug print line before Pass 2.

Pass 2 — compute:

```
discount_decimal = discount_percent / 100
revenue = retail_price * quantity_sold
discounted_revenue = revenue * (1 - discount_decimal)
gross_margin = (retail_price - unit_cost) / retail_price
net_profit = discounted_revenue - (unit_cost * quantity_sold)
```

Line-by-line explanation:

- `discount_decimal = discount_percent / 100` — this is the exact conversion flagged as missing above: `15` becomes `0.15`. Call back to the Exercise 4 mistake pattern (forgetting this step) and to the reflection question at the end of the lab, which asks students to describe exactly this bug.
- `revenue = retail_price * quantity_sold` — same formula as Exercise 1, now using live user input instead of hardcoded numbers.
- `discounted_revenue = revenue * (1 - discount_decimal)` — read the business logic aloud: if the discount is 15%, the customer pays 85% of full price, so multiply revenue by `(1 - 0.15) = 0.85`. The parentheses around `1 - discount_decimal` are required for the same reason as Exercise 2 — without them, order of operations would multiply `discount_decimal` by `revenue` first, then subtract that product from `1`, which is a nonsense number.
- `gross_margin = (retail_price - unit_cost) / retail_price` — identical formula to Exercise 2, reused here at the per-unit level (margin doesn't depend on quantity).
- `net_profit = discounted_revenue - (unit_cost * quantity_sold)` — total cost of goods sold is `unit_cost * quantity_sold`; net profit is what's left of discounted revenue after covering that cost. The parentheses here are not strictly required (`*` already binds tighter than `-`), but include them anyway and say why: **explicit parentheses that aren't strictly necessary are still good practice** — they make the formula readable at a glance without anyone needing to recall precedence rules. This is a deliberate style point, not a correctness one.

Pass 3 — formatted report:

```
print(f"Product: {product_name}")
print(f"Revenue (before discount): ${revenue:,.2f}")
print(f"Discounted revenue: ${discounted_revenue:,.2f}")
print(f"Gross margin: {gross_margin:.1%}")
print(f"Net profit: ${net_profit:,.2f}")
```

Every format spec here reuses exactly what students learned in Exercises 1 and 2 — call that out explicitly ("nothing new in this block, only application").

Run the full script with sample input `Wireless Mouse`, `29.99`, `11.50`, `340`, `15`. **Expected output:**

```
Product: Wireless Mouse
Revenue (before discount): $10,196.60
Discounted revenue: $8,667.11
Gross margin: 61.7%
Net profit: $4,757.11
```

Common student mistakes to watch for:

- Forgetting `float()`/`int()` around `input()` entirely, causing `retail_price * quantity_sold` to either crash (`TypeError: can't multiply sequence by non-int of type 'str'`) or, worse, silently do string repetition if only one side got converted. Walk the room during Pass 1 specifically for this.
- Using `discount_percent` directly instead of `discount_decimal` in the `discounted_revenue` line — produces a wildly wrong (usually negative or huge) number rather than a crash. This is the flagship "silent wrong answer" of the whole lab; if you see it, don't just fix it — ask the student what the output *should* look like and let them notice the number is implausible on their own first.
- Typing a comma or dollar sign when prompted (e.g. `$29.99`) — `float()` cannot parse that and raises `ValueError`. Worth a 10-second warning before students test their own script.

Check for understanding: "Why does `product_name` not need `float()` or `int()` around it, but every other input line does?" (Because it's meant to stay text — converting it would either error out or be meaningless. Get someone to state that conversion is a deliberate choice based on what the value represents, not a rule applied to every `input()` call.)

6.8 Exercise 7 — Packaging Problem (0:55–1:07, 12 min)

Teaching goal: Applying `//` and `%` together to a real logistics scenario — this is the payoff exercise for Exercise 5's prediction work.

Say to the class:

> *"A warehouse ships in boxes of 24. Someone orders 100 items. How many full boxes, and how many are left over? You already know how to answer this — you did it by hand at the end of Exercise 5."*

Live-code this:

```
# --- Exercise 7 ---
items_ordered = int(input("Items ordered: "))
box_size = 24
full_boxes = items_ordered // box_size
loose_items = items_ordered % box_size
exact_fit = loose_items == 0
print(f"Full boxes: {full_boxes}")
print(f"Loose items: {loose_items}")
print(f"Exact fit: {exact_fit}")
```

Line-by-line explanation:

- `items_ordered = int(input(...))` — same `int(input(...))` pattern from Exercise 6; ask the room to name it before you explain it, they should recognize it now.
- `box_size = 24` — fixed, not user input, since it's a warehouse constant, not something that varies per order.
- `full_boxes = items_ordered // box_size` — floor division from Exercise 5, now doing real work: how many complete groups of 24 fit into `items_ordered`.
- `loose_items = items_ordered % box_size` — modulo, again from Exercise 5: whatever's left after taking out as many full boxes as possible. Re-state the identity from Exercise 5 with real numbers once the output is on screen: `4 × 24 + 4 = 100`.
- `exact_fit = loose_items == 0` — this is a **new use of ==**, the equality comparison operator, distinct from the assignment = used everywhere else in this script. Say explicitly and slowly: one equals sign assigns a value into a variable; two equals signs asks "are these two things equal?" and produces a `bool`. This distinction trips up nearly every student at some point this semester — spend 20 real seconds on it here rather than assuming Exercise 3's > example already covered it.

Run it with `items_ordered = 100`. Expected output:

```
Full boxes: 4
Loose items: 4
```

```
Exact fit: False
```

Immediately re-run with a number that divides evenly, e.g. 96, live:

```
Full boxes: 4
Loose items: 0
Exact fit: True
```

This second run is important — it's the only case in the whole lab where students see `exact_fit` actually flip to `True`, and it confirms `== 0` is doing real comparison work, not just always returning `False`.

Common student mistakes to watch for:

- Writing `exact_fit = loose_items = 0` (single `=`) by accident — this **assigns** `0` to both `loose_items` and `exact_fit`, silently overwriting the real remainder with `0` and making `exact_fit` hold the integer `0` instead of a boolean. Because Python treats `0` as "falsy," `print(f"Exact fit: {exact_fit}")` would print `Exact fit: 0`, not `Exact fit: False` — a good diagnostic tell if a student's output looks almost right but not quite.
- Reversing the order — `box_size // items_ordered` — which runs without error but answers a different, wrong question. Ask a comprehension question rather than immediately pointing it out: "What does that number even represent?"

Check for understanding: "A shift is 8 hours long. An employee has worked 53 hours this week. Using the same two operators, what expression tells you how many *full* shifts that is, and what expression tells you the *leftover* partial hours?" (`53 // 8` → 6 full shifts; `53 % 8` → 5 leftover hours. This is the built-in extra practice example for this exercise — use it here if you have a couple of spare minutes, or hold it for the wrap-up if the room needs one more rep of the same idea in a new context.)

6.9 Stretch A — Break-Even Calculator (as time allows)

Teaching goal: Combining `//` and `%` in a genuinely useful financial formula — how many units does a business need to sell before it stops losing money.

Frame it, live-code it if you have time, otherwise assign it as optional take-home practice:

```
# --- Stretch A ---
fixed_costs = int(input("Fixed costs (rent, salaries, etc.): "))
variable_cost_per_unit = float(input("Variable cost per unit: "))
retail_price_per_unit = float(input("Retail price per unit: "))

contribution_margin = retail_price_per_unit - variable_cost_per_unit
break_even_units = fixed_costs // contribution_margin
remainder = fixed_costs % contribution_margin

print(f"Break-even quantity: {break_even_units:.0f} whole units, "
      f"plus ${remainder:,.2f} still uncovered at that quantity")
print(f"Exact (fractional) break-even: "
      f"{fixed_costs / contribution_margin:.2f} units")
```

Line-by-line explanation (abbreviated — same operators as Exercises 5–7, new formula):

- `contribution_margin = retail_price_per_unit - variable_cost_per_unit` — how much of each unit's price is left after covering the *variable* cost of making it; this is what pays down the *fixed* costs, one unit at a time.
- `break_even_units = fixed_costs // contribution_margin` — how many whole units it takes for accumulated contribution margin to cover fixed costs.
- `remainder = fixed_costs % contribution_margin` — how much fixed cost is still uncovered right at that whole-unit boundary — this is why the business actually needs to sell *one more* unit than the floor-division answer to be truly profitable, which is a genuinely good discussion point if time allows.

Run with `fixed_costs = 12500`, `variable_cost_per_unit = 8`, `retail_price_per_unit = 20`. **Expected output:**

```
Break-even quantity: 1041 whole units, plus $8.00 still uncovered at that quantity
Exact (fractional) break-even: 1041.67 units
```

6.10 Stretch B — Operator Precedence Trap (as time allows)

Teaching goal: This exercise is the direct payoff of every "the parentheses are required here" aside from the last hour. If you only have time for one stretch exercise, pick this one over Stretch A — it's the more important idea and does not require re-explaining a new formula.

Say to the class:

> *"Five expressions. For each, I'll show you the version without parentheses and the version with — and you'll see they give* different numbers. *Neither one crashes. That's the whole danger."*

Work through as many of these five as time allows, live, showing both versions:

1. Average of three margins

```
a, b, c = 10, 20, 90
wrong = a + b + c / 3
right = (a + b + c) / 3
```

`wrong` = `60.0` (only `c` gets divided by 3, because `/` binds tighter than `+`). `right` = `40.0`, the true average.

2. Margin formula

```
price, cost = 600, 320
wrong = price - cost / price
right = (price - cost) / price
```

`wrong` = `599.47` — division happens before subtraction, so this computes `price - (cost / price)`, a number that looks superficially plausible as a dollar figure but is not a margin at all. `right` = `0.4667`, the correct 46.7% margin.

3. Logical flag (`or` / `and`)

```
price, margin = 15, 0.2
wrong = price < 20 or price > 200 and margin < 0.15
right = (price < 20 or price > 200) and margin < 0.15
```

`wrong` = `True` (`and` binds tighter than `or`, so this evaluates as `price < 20 or (price > 200 and margin < 0.15)` — the low price alone triggers the flag). `right` = `False` (grouping the "low or high price" check together and requiring thin margin *in addition*). Same variables, opposite conclusions — flag this as the most dangerous of the five, because a boolean result gives no numeric hint that anything is wrong.

4. Reserved-stock boxing problem

```
items, reserved, box_size = 100, 15, 24
wrong = items - reserved // box_size
right = (items - reserved) // box_size
```

`wrong` = `100` (`reserved // box_size` is `0`, since 15 < 24, so nothing is subtracted at all). `right` =

`3` (subtract the 15 reserved items first, *then* box the remaining 85).

5. Compound growth

```
principal, rate, years = 1000, 0.05, 3
wrong = principal * 1 + rate ** years
right = principal * (1 + rate) ** years
```

`wrong` = `1000.000125` (`**` binds before `*`, and this computes `(principal * 1) + (rate ** years)` — nowhere close to compound growth). `right` = `1157.63`, correct 5% compounded over 3 years.

For each one you work through, require the same three-part answer the exercise asks for: (a) the unparenthesized expression and its result, (b) the parenthesized expression and its result, (c) one sentence on which is correct and why. Do not just show the numbers — make students articulate the *why*, since that's the actual submitted deliverable.

7 Wrap-Up (last ~5 minutes of the 1:07–1:15 block)

Review the reflection questions out loud (full text is on the student lab page) — do not answer them for the class, but preview what a strong answer looks like:

1. *Forgetting to divide `discount_percent` by 100* — this does **not** crash Python. It silently produces a nonsensical `discounted_revenue`, often negative. Reinforce: this is the exact "silent wrong answer" category the whole lab has been building toward.
2. *Another modulo scenario* — accept anything with a real "how many full groups, how much left over" structure: paying employees in fixed-size batches, filling shipping pallets, splitting a bill among tables at a restaurant.
3. *Least confident concept from Modules 1–4* — there's no wrong answer here; the point is that this question is genuinely diagnostic for you as the instructor. If several students name the same concept, that's a signal for next week's office hours or a two-minute recap at the start of Module 05.

Review the submission checklist together, on screen:

- ☐ File is named `calculator.py`
- ☐ Contains all seven exercises (1–7) in one file, in order
- ☐ Each exercise has a `# --- Exercise N ---` comment above it
- ☐ All numeric output uses an appropriate f-string format spec (`,.2f` for currency, `.1%` for percentages)
- ☐ Script runs top to bottom with no errors when given valid input

Preview Module 05: "Every boolean we made today — `is_premium`, `product_a_wins`, `exact_fit` — just sat there as a printed value. Next module, we finally get to *branch* on them: `if`/`elif`/`else`. Today's `and`/`or` work is the direct prerequisite."

8 Appendix A — Full Answer Key (`calculator.py`)

Use this to sanity-check your own live-coded version before class, or to hand out after grading if students want a reference.

```python
# calculator.py
# ISM2411 Module 04 Lab — Revenue, Margin & Discount Calculator

# --- Exercise 1 ---
price = 50
quantity = 12
revenue = price * quantity
print(f"Revenue: ${revenue:,.2f}")

# --- Exercise 2 ---
cost = 32
price = 50
margin = (price - cost) / price
print(f"Margin: {margin:.1%}")

# --- Exercise 3 ---
price_a, cost_a = 50, 32
price_b, cost_b = 80, 60
margin_a = (price_a - cost_a) / price_a
margin_b = (price_b - cost_b) / price_b
product_a_wins = margin_a > margin_b
print(f"Product A margin: {margin_a:.1%}")
print(f"Product B margin: {margin_b:.1%}")
print(f"Product A wins: {product_a_wins}")

# --- Exercise 4 ---
price = 120
margin = 0.35
is_premium = price > 100 and margin > 0.3
print(f"is_premium = {is_premium}")

# --- Exercise 5 ---
print(17 / 5)    # true division  -> 3.4
print(17 // 5)   # floor division -> 3
print(17 % 5)    # modulo         -> 2

# --- Exercise 6 ---
```

```
product_name = input("Product name: ")
retail_price = float(input("Retail price: "))
unit_cost = float(input("Unit cost: "))
quantity_sold = int(input("Quantity sold: "))
discount_percent = float(input("Discount percent (e.g. 10 for 10%): "))

discount_decimal = discount_percent / 100
revenue = retail_price * quantity_sold
discounted_revenue = revenue * (1 - discount_decimal)
gross_margin = (retail_price - unit_cost) / retail_price
net_profit = discounted_revenue - (unit_cost * quantity_sold)

print(f"Product: {product_name}")
print(f"Revenue (before discount): ${revenue:,.2f}")
print(f"Discounted revenue: ${discounted_revenue:,.2f}")
print(f"Gross margin: {gross_margin:.1%}")
print(f"Net profit: ${net_profit:,.2f}")

# --- Exercise 7 ---
items_ordered = int(input("Items ordered: "))
box_size = 24
full_boxes = items_ordered // box_size
loose_items = items_ordered % box_size
exact_fit = loose_items == 0
print(f"Full boxes: {full_boxes}")
print(f"Loose items: {loose_items}")
print(f"Exact fit: {exact_fit}")
```

Sample run for Exercise 6 (input: `Wireless Mouse`, `29.99`, `11.50`, `340`, `15`):

```
Product: Wireless Mouse
Revenue (before discount): $10,196.60
Discounted revenue: $8,667.11
Gross margin: 61.7%
Net profit: $4,757.11
```

Sample run for Exercise 7 (input: `100`):

```
Full boxes: 4
Loose items: 4
Exact fit: False
```

9 Appendix B — Extra Practice (only if the class finishes early)

This lab's seven required exercises plus two stretch challenges fill the full 75 minutes at a normal teaching pace, so treat this appendix as a release valve rather than core content — use it only if a section moves unusually fast, or assign individual items to early finishers while the rest of the class catches up.

Extra 1 — Different numbers, Exercise 1/2 pattern. A product has `price = 84` and `cost = 61`. Have students compute and print revenue for `quantity = 27` and the margin, using the exact same two format specs from Exercises 1–2. (Revenue: `$2,268.00`. Margin: `27.4%`.)

Extra 2 — Different numbers, Exercise 7 pattern. A concert venue seats people in rows of 18. `212` tickets were sold. How many full rows, how many people in the partial row, and is it an exact fit? (`11` full rows, `14` left over, `Exact fit: False`.) This is the same shift/box-packing idea as the Exercise 7 check-for-understanding question — use whichever framing (warehouse, shifts, seating) the room hasn't already seen.

Extra 3 — One more precedence trap, in the style of Stretch B. A store takes a 20% employee discount off retail price, then adds 7% sales tax on the discounted price. `retail_price = 100`.

```
wrong = 100 - 0.20 * 100 + 0.07 * 100 - 0.20 * 100
right = (100 - 0.20 * 100) * 1.07
```

Have students predict both before running. (`wrong` = `67.0` — subtracting the discount amount *twice* because the un-grouped expression re-applies `0.20 * 100` as a second standalone term instead of building on the already-discounted price. `right` = `85.6`, the correct discounted-then-taxed price.) Ask the room why `wrong` is still a plausible-looking dollar figure rather than an obvious error — that's the point: it's low enough to pass a sloppy sanity check, which is exactly why this category of bug is dangerous. This is a good closer if you want one more "looks right but isn't" example before moving on.